Reviewer Pack — Minimal Rank of Quasi-Plurality Matrices vs SAT Satisfiabili…

Entry 432931f4b53b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 09:52:49 UTC

Minimal Rank of Quasi-Plurality Matrices vs SAT Satisfiability

Entry ID: `432931f4b53b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 09:52:49 UTC

1. Conjecture

Field A (mathematical branch): Quasi-Plurality Theory (Algebraic Combinatorics) **Field B** (complexity object): Complexity Theory: SAT Satisfiability

Statement:

> {'p1': 'For every satisfiable CNF formula with n variables, the minimal rank of its corresponding quasi-plurality matrix is at most cn for some constant c.', 'p2': 'There exists a polynomial-time constructive mapping from a satisfiable CNF instance to its quasi-plurality matrix.', 'p3': 'For every unsatisfiable CNF formula with n variables, the minimal rank of its corresponding quasi-plurality matrix is at least d^n for some constant d.'}

Rationale (proposer's reasoning):

> {'p1': 'Quasi-plurality matrices capture combinatorial properties that could be relevant to the complexity of SAT.', 'p2': 'The connection between quasi-plurality theory and SAT has not been extensively explored, offering a novel angle for complexity lower bounds.', 'p3': 'Previous work on quasi-plurality matrices mainly focused on arithmetic circuits, suggesting potential relevance to Boolean circuit complexity.'}

Taxonomy category: `QuasiPluralitySAT` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `672d5246214443ca`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated CNF formulas with n variables, the minimal rank of the corresponding quasi-plurality matrix meets the following conditions: for satisfiable formulas, the mean minimal rank across seeds is ≤ cn and support_fraction ≥ 0.8; for unsatisfiable formulas, the mean minimal rank across seeds is ≥ d^n.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `minimal rank quasi-plurality matrices cnf satisfiability` - `quasi-plurality theory mapping sat satisfiability polynomial-time` - `unsatisfiable cnf formulas minimal rank quasi-plurality matrix d^n`

Top relevant hits considered: - [http://arxiv.org/abs/2006.02374v2] On tensor rank and commuting matrices - [http://arxiv.org/abs/1503.04226v2] Symmetric Hadamard matrices of order 116 and 172 exist - [http://arxiv.org/abs/2403.03513v3] Spectrum of random centrosymmetric matrices; CLT and Circular law - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/1505.03340v2] HordeSat: A Massively Parallel Portfolio SAT Solver - [http://arxiv.org/abs/2605.09347v1] Dsat: A Native SAT Solver for Discrete Logic - [http://arxiv.org/abs/1203.3706v2] On the Complexity of Computing Minimal Unsatisfiable LTL formulas - [http://arxiv.org/abs/0806.1148v2] Unsatisfiable CNF Formulas need many Conflicts

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Helper functions for Gaussian elimination and matrix operations
def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        # Swap rows
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate lower entries
        pivot = matrix[i][i]
        for j in range(i, n):
            matrix[i][j] /= pivot
        for k in range(i+1, n):
            factor = matrix[k][i]
            for j in range(i, n):
                matrix[k][j] -= factor * matrix[i][j]
```

```
def rank(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [0] * (n - len(row)) for row in matrix]
    gaussian_elimination(augmented_matrix)
    rank = 0
    for row in augmented_matrix:
        if any(x != 0 for x in row):
            rank += 1
    return rank

def quasi_plurality_matrix(cnf):
    n = len(cnf[0])
    matrix = [[0] * (2*n) for _ in range(2*n)]

    for clause in cnf:
        literals = set()
        for literal in clause:
            if literal > 0:
                literals.add(literal)
            else:
                literals.add(-literal)

        for lit1 in literals:
            for lit2 in literals:
                matrix[lit1-1][lit2-1] += 1

    return matrix

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random CNF formula
    n = random.randint(5, 30)
    cnf = []
    for _ in range(n):
        clause = [random.choice([-i, i]) for i in range(1, n+1)]
        cnf.append(clause)

    # Compute the quasi-plurality matrix
    try:
        matrix = quasi_plurality_matrix(cnf)
    except IndexError:
        return {
            "metric_name": "rank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    # Determine the minimal rank
    min_rank = rank(matrix)

    return {
        "metric_name": "rank",
        "metric_value": min_rank,
```

```
        "instances_tested": 1,
        "conjecture_holds": False,  # This will be updated later
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    # Compute mean and standard deviation of metric_value
    if all(result["metric_value"] is not None for result in results):
        values = [result["metric_value"] for result in results]
        mean = sum(values) / len(values)
        std_dev = math.sqrt(sum((x - mean) ** 2 for x in values) / len(values))

        # Compute fraction of seeds where conjecture_holds
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction
        else:
            print(f"RESULT: FALSIFIED counterexample='' first_failing_seed={seeds[results.index(ne
    else:
        print("RESULT: INCONCLUSIVE reason=metric_value_none")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a355a25.py", line 106, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a355a25.py", line 90, in run_trial
    min_rank = rank(matrix)
               ^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a355a25.py", line 42, in rank
    gaussian_elimination(augmented_matrix)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a355a25.py", line 33, in gaussian_elim
    matrix[i][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed due to a division by zero error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with a larger sample size of CNF formulas to ensure robustness.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14769
2	preregistration	ollama_remote	glm4:latest	0	0	9846
3	novelty	ollama_remote	glm4:latest	0	0	8261
4	novelty	ollama_remote	glm4:latest	0	0	9731
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18580
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10107
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10347
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10698
9	judge	ollama_remote	glm4:latest	0	0	11489

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 103827 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/432931f4b53b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/432931f4b53b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/432931f4b53b.tar.gz` (if generated)